

PRAKTIKUM MACHINE LEARNING

Bab 10

Clustering

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

2 Oktober 2026

Daftar Isi

Clustering	3
Ringkasan Konsep	4
10.12 Latihan Praktikum (Modul Bab 10).....	4
Percobaan Mandiri.....	11
Kesimpulan Bab 10	17

Tools yang Digunakan

Python 3.12.3 • numpy 2.5.3 • pandas 3.0.6 • matplotlib 3.11.2 • seaborn 0.13.2 • scikit-learn 1.9.1 • imbalanced-learn 0.14.2

Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab, jalankan sel di bawah ini dulu untuk memastikan semua paket yang dipakai sudah tersedia. Kalau saya menjalankannya di Jupyter lokal, sel ini tidak mengubah apa-apa karena semua paketnya sudah terpasang di environment.

In [1] - Kode Program

```
1 # Sel 0 - Persiapan awal (jalan otomatis paling awal saat Run All).
2 # Semua paket di bawah SUDAH tersedia di environment,
3 # sel ini hanya memastikan import-nya tidak error.
4 try:
5     import sklearn
6     import pandas
7     import numpy
8     import matplotlib
9     print("Paket siap.")
10 except ImportError as e:
11     print("Paket belum lengkap:", e)
```

Paket siap.

Praktikum Machine Learning: Bab 10

Clustering

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Catatan: notebook ini saya jalankan di Google Colab / Jupyter Notebook. Bab 10 ini mengerjakan **Latihan Praktikum Modul Bab 10 bagian 10.12**, memakai **data sintetis** pelanggan (sebagai pengganti dataset Mall Customer Segmentation dari Kaggle, sesuai izin modul) dari `sklearn.datasets.make_blobs`. Tidak perlu file CSV apa pun.

Ringkasan Konsep

- **Clustering itu unsupervised learning:** model mencari kelompok-kelompok sendiri dari pola datanya, tanpa diberi label target seperti pada klasifikasi.
- **K-Means:** pilih k titik centroid, tiap data ditempelkan ke centroid terdekat, centroid dihitung ulang dari rata-rata anggotanya, lalu diulang sampai tidak berubah lagi.
- **Elbow method:** plot nilai inertia (total kuadrat jarak tiap titik ke centroid clusternya) untuk berbagai nilai k. k optimal ada di "siku" grafik, yaitu titik tempat penurunan inertia mulai melandai.
- **Silhouette score:** mengukur seberapa cocok tiap titik dengan clusternya sendiri dibanding cluster tetangga terdekat. Nilainya dari -1 sampai 1, makin dekat ke 1 berarti pemisahan clusternya makin bagus.
- **Agglomerative clustering** menggabungkan titik atau cluster yang paling mirip secara bertahap dari bawah ke atas (hierarchical). **DBSCAN** mengelompokkan titik-titik yang padat bertetangga dan menandai titik yang sepi sebagai noise, sehingga cocok untuk cluster yang bentuknya tidak beraturan.

10.12 Latihan Praktikum (Modul Bab 10)

Di bab ini saya mengerjakan dua Latihan Praktikum dari modul bagian 10.12: pertama menentukan jumlah cluster yang optimal, lalu membandingkan tiga algoritma clustering pada dataset yang sama.

Praktikum 1 - Menentukan k Optimal

Tujuan: menentukan jumlah cluster optimal menggunakan Elbow Method dan Silhouette Score.

Instruksi (modul): menggunakan dataset pelanggan (modul membolehkan data sintetis), buat plot Elbow Method dan Silhouette Score untuk $k=2$ hingga $k=10$, kemudian tentukan k optimal beserta justifikasinya.

Saya memakai data sintetis pelanggan dari `make_blobs` dengan 5 pusat cluster dan 1000 baris. Dua fiturnya saya beri nama "Annual Income" dan "Spending Score" supaya terasa seperti data segmentasi pelanggan mal.

In [2] - Kode Program

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.datasets import make_blobs
5 from sklearn.cluster import KMeans
6 from sklearn.metrics import silhouette_score
7
8 # Data sintetis pelanggan: 1000 baris, 5 pusat cluster, 2 fitur.
9 X, _ = make_blobs(n_samples=1000, centers=5, n_features=2,
10                  cluster_std=1.2, random_state=42)
11 df = pd.DataFrame(X, columns=["Annual Income", "Spending Score"])
12 print("Bentuk data:", df.shape)
13 print(df.head())
14
15 # Elbow method + silhouette score untuk k = 2 sampai 10.
16 ks = list(range(2, 11))
17 inertias, silhouettes = [], []
18 for k in ks:
19     km = KMeans(n_clusters=k, n_init=10, random_state=42)
20     labels = km.fit_predict(X)
21     inertias.append(km.inertia_)
22     silhouettes.append(silhouette_score(X, labels))
23     print(f"k={k}: inertia={km.inertia_:.1f}, silhouette={silhouettes[-1]:.3f}")
24
25 # Satu figure, dua subplot berdampingan.
26 fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
27 axes[0].plot(ks, inertias, marker="o")
28 axes[0].set_xlabel("k (jumlah cluster)")
29 axes[0].set_ylabel("Inertia")
30 axes[0].set_title("Elbow Method")
31 axes[0].set_xticks(ks)
32 axes[1].plot(ks, silhouettes, marker="o", color="green")
33 axes[1].set_xlabel("k (jumlah cluster)")
34 axes[1].set_ylabel("Silhouette Score")
35 axes[1].set_title("Silhouette Score vs k")
36 axes[1].set_xticks(ks)
37 plt.tight_layout()
38 plt.show()
39
40 k_optimal = ks[int(np.argmax(silhouettes))]
41 print("k dengan silhouette score tertinggi:", k_optimal)

```

Bentuk data: (1000, 2)

	Annual Income	Spending Score
0	5.096116	2.705873
1	2.950865	1.039790
2	-5.953589	-10.298361
3	5.311627	3.270107
4	1.252522	4.679759

k=2: inertia=29580.7, silhouette=0.578

k=3: inertia=8198.6, silhouette=0.679

k=4: inertia=3961.2, silhouette=0.687

k=5: inertia=2633.8, silhouette=0.629

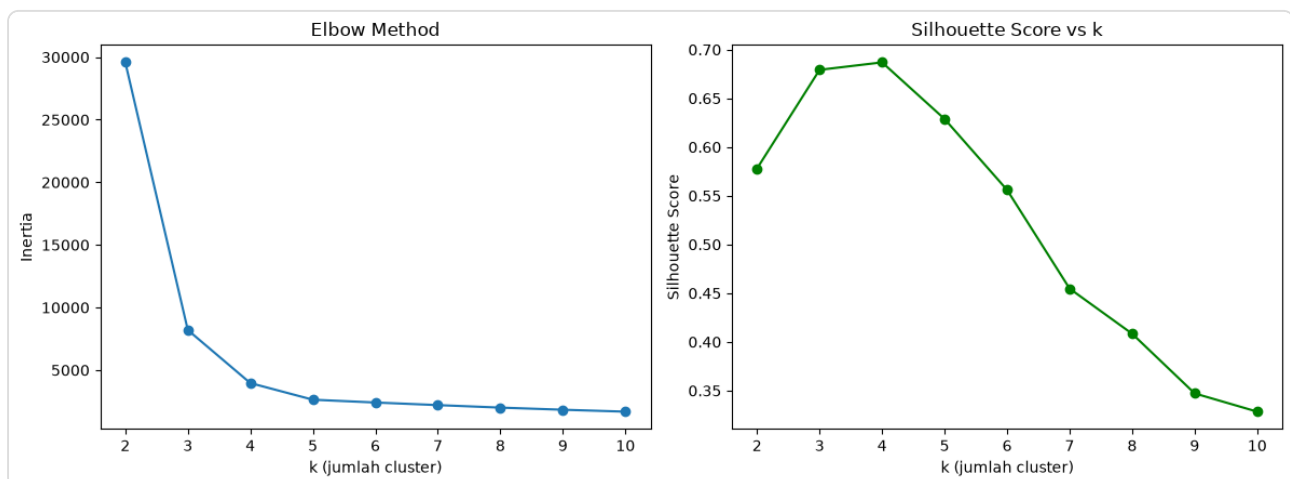
k=6: inertia=2410.3, silhouette=0.556

k=7: inertia=2202.1, silhouette=0.455

k=8: inertia=2002.3, silhouette=0.408

k=9: inertia=1833.0, silhouette=0.347

k=10: inertia=1679.4, silhouette=0.328



Gambar 1

k dengan silhouette score tertinggi: 4

Penjelasan outputnya:

Datanya terbaca 1000 baris dengan 2 kolom, "Annual Income" dan "Spending Score", sesuai yang saya buat dari `make_blobs`. Dari hasil loop k=2 sampai k=10, inertia turun sangat tajam di awal: 29580.7 (k=2), 8198.6 (k=3), 3961.2 (k=4), 2633.8 (k=5), lalu melandai setelahnya (2410.3 di k=6, 2202.1 di k=7, dan seterusnya). Di grafik elbow, "siku" nya terlihat jelas di k=5, karena penurunan dari k=4 ke k=5 masih besar (sekitar 1327) sedangkan dari k=5 ke k=6 tinggal sekitar 223.

Silhouette score tertinggi ada di $k=4$ (0.687), disusul $k=3$ (0.679) dan $k=5$ (0.629). Setelah $k=5$ skornya anjlok terus sampai 0.328 di $k=10$, yang artinya memaksakan terlalu banyak cluster justru merusak pemisahannya.

Justifikasi k optimal: saya pilih $k=5$. Elbow method menunjuk $k=5$ dengan tegas, silhouette di $k=5$ masih bagus (0.629, tertinggi ketiga dan jauh di atas $k=6$ ke atas), dan angka ini juga cocok dengan 5 pusat cluster yang saya pakai saat membuat datanya. Jadi $k=5$ adalah titik temu kedua metode.

Praktikum 2 - Segmentasi dengan Tiga Algoritma

Tujuan: membandingkan hasil clustering ketiga algoritma pada dataset yang sama.

Instruksi (modul, Praktikum 10.7): kode di modul hanya berupa komentar, jadi saya implementasikan langsung seperti ini:

```
# Terapkan K-Means, Hierarchical (Agglomerative),  
# dan DBSCAN pada dataset yang sama.  
# Bandingkan jumlah cluster yang dihasilkan,  
# Silhouette Score, dan visualisasikan ketiganya  
# dalam satu figure berdampingan (subplot). Tuliskan analisis  
# mengenai algoritma mana  
# yang menghasilkan segmentasi paling masuk akal secara bisnis  
# untuk kasus Anda.
```

Saya memakai dataset X yang sama seperti Praktikum 1. Untuk DBSCAN saya pakai $\text{eps}=0.9$ dan $\text{min_samples}=10$, dan silhouette score-nya saya hitung di luar titik noise (label -1).

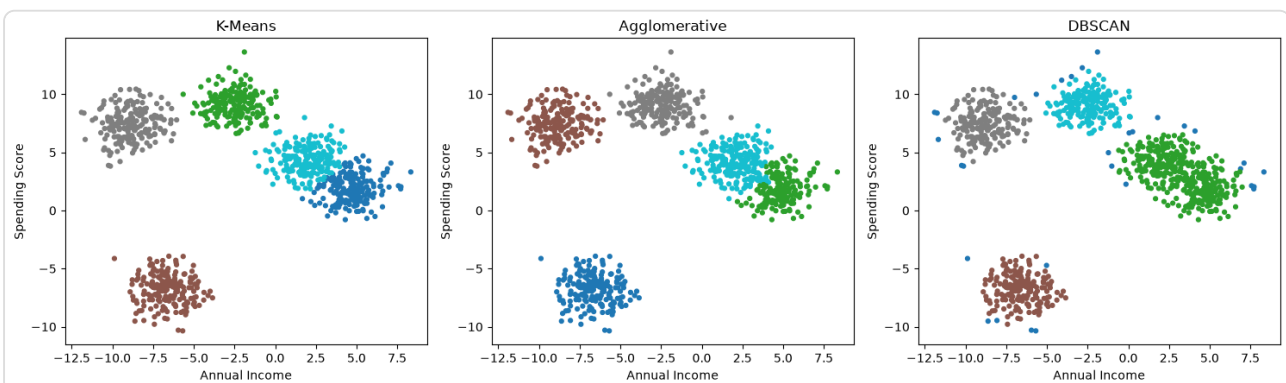
In [3] - Kode Program

```

1 from sklearn.cluster import AgglomerativeClustering, DBSCAN
2
3 # Terapkan ketiga algoritma pada dataset yang sama.
4 hasil_label = {
5     "K-Means": KMeans(n_clusters=5, n_init=10, random_state=42).fit_predict(X),
6     "Agglomerative": AgglomerativeClustering(n_clusters=5).fit_predict(X),
7     "DBSCAN": DBSCAN(eps=0.9, min_samples=10).fit_predict(X),
8 }
9
10 # Bandingkan jumlah cluster dan silhouette score tiap algoritma.
11 print(f"{'Algoritma':<14}{'Cluster':<9}{'Noise':<8}{'Silhouette'}")
12 for nama, labels in hasil_label.items():
13     tanpa_noise = labels[labels != -1]
14     n_cluster = len(set(tanpa_noise))
15     n_noise = int((labels == -1).sum())
16     sil = silhouette_score(X[labels != -1], tanpa_noise) if n_cluster > 1 else
float("nan")
17     print(f"{'nama':<14}{'n_cluster':<9}{'n_noise':<8}{'sil':.3f}")
18
19 # Visualisasikan ketiganya dalam satu figure berdampingan.
20 fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
21 for ax, (nama, labels) in zip(axes, hasil_label.items()):
22     ax.scatter(X[:, 0], X[:, 1], c=labels, cmap="tab10", s=12)
23     ax.set_xlabel("Annual Income")
24     ax.set_ylabel("Spending Score")
25     ax.set_title(nama)
26 plt.tight_layout()
27 plt.show()

```

Algoritma	Cluster	Noise	Silhouette
K-Means	5	0	0.629
Agglomerative	5	0	0.623
DBSCAN	4	31	0.699



Gambar 2

Penjelasan outputnya:

Tabel perbandingannya menunjukkan K-Means dan Agglomerative sama-sama menghasilkan tepat 5 cluster tanpa noise, dengan silhouette score yang hampir identik: 0.629 untuk K-Means dan 0.623 untuk Agglomerative. DBSCAN menemukan 4 cluster dengan 31 titik ditandai sebagai noise, dan silhouette score di luar noise-nya 0.699, paling tinggi di antara ketiganya.

Dari scatter plot berdampingan terlihat K-Means dan Agglomerative memetakan kelima gugus pelanggan dengan rapi dan konsisten. DBSCAN menggabungkan dua gugus yang berdekatan menjadi satu cluster dan membuang 31 titik sebagai noise, sehingga segmentasinya lebih kasar.

Analisis: untuk kasus segmentasi pelanggan ini, **K-Means paling masuk akal secara bisnis**. Alasannya: setiap pelanggan harus masuk ke satu segmen agar bisa ditindaklanjuti tim marketing, dan K-Means menempatkan semua 1000 pelanggan ke 5 segmen yang jelas tanpa ada yang dibuang sebagai noise. Agglomerative hasilnya nyaris sama bagusnya, jadi bisa jadi alternatif. DBSCAN memang punya silhouette tertinggi, tapi ia mengorbankan satu segmen dan membuang 31 pelanggan sebagai noise, sehingga kurang cocok dipakai untuk segmentasi pelanggan yang butuh seluruh data terpetakan.

Percobaan Mandiri

Di bawah ini saya tambahkan tiga percobaan mandiri di luar dua Latihan Praktikum dari modul, untuk menguji beberapa hal yang belum dibahas di atas: efek strategi inisialisasi KMeans, pengaruh scaling fitur terhadap hasil clustering, dan sensitivitas DBSCAN terhadap parameter eps.

Percobaan 1 - Inisialisasi KMeans: random vs k-means++

Tujuan: membandingkan kestabilan hasil KMeans antara inisialisasi pusat cluster yang acak (`init="random"`) dan yang cerdas (`init="k-means++"`).

Saya membuat data `make_blobs` dengan 500 titik dan 4 cluster, lalu menjalankan KMeans masing-masing 10 kali dengan seed berbeda (`n_init=1` supaya tiap seed benar-benar satu run) dan melihat rata-rata serta standar deviasi inertia-nya. Kalau sebuah metode init bagus, rata-ratanya harus rendah dan std-nya kecil (hasilnya konsisten di semua seed).

In [4] - Kode Program

```
1 import numpy as np
2 from sklearn.datasets import make_blobs
3 from sklearn.cluster import KMeans
4
5 # Data: 500 titik, 4 cluster (dibuat mandiri, tidak tergantung sel lain)
6 X_pm1, _ = make_blobs(n_samples=500, centers=4, random_state=42)
7
8 # Jalankan KMeans 10 kali dengan seed berbeda untuk tiap metode init
9 for metode in ["random", "k-means++"]:
10     inertia = [
11         KMeans(n_clusters=4, init=metode, n_init=1,
12 random_state=s).fit(X_pm1).inertia_
13     ]
14     print(f"init={metode:<10} rata-rata inertia={np.mean(inertia):.2f} "
15           f"std={np.std(inertia):.2f} (min={min(inertia):.2f})")
```

```
init=random      rata-rata inertia=1904.10 std=1169.89 (min=948.89)
```

```
init=k-means++   rata-rata inertia=948.89 std=0.00 (min=948.89)
```

Penjelasan outputnya:

Hasilnya sesuai dugaan. Dengan `init="random"`, rata-rata inertia-nya 1904.10 dengan std 1169.89: artinya hasilnya sangat tergantung keberuntungan seed, ada run yang bagus (inertia 948.89) tapi banyak juga yang nyangkut di minimum lokal dengan inertia jauh lebih besar. Sementara `init="k-means++"` menghasilkan rata-rata inertia 948.89 dengan std 0.00, artinya semua 10 seed dapat nilai optimum yang sama.

Kesimpulan: inisialisasi k-means++ jauh lebih stabil daripada random. Karena k-means++ memilih centroid awal yang saling berjauhan, KMeans tidak lagi sensitif terhadap seed dan hampir selalu menemukan solusi terbaik. Ini alasan kenapa k-means++ jadi init default di scikit-learn.

Percobaan 2 - Scaling Fitur: dengan vs tanpa StandardScaler

Tujuan: membuktikan bahwa fitur dengan skala besar bisa mendominasi jarak Euclidean di KMeans kalau datanya tidak di-scaling.

Saya membuat data 2D dengan skala yang timpang: fitur pertama kisarannya 0-1, fitur kedua saya kalikan 1000 sehingga kisarannya 0-1000. Karena saya tahu label asli cluster-nya, saya bisa menilai kualitas clustering dengan Adjusted Rand Index (ARI): 1.0 berarti sama persis dengan struktur asli datanya.

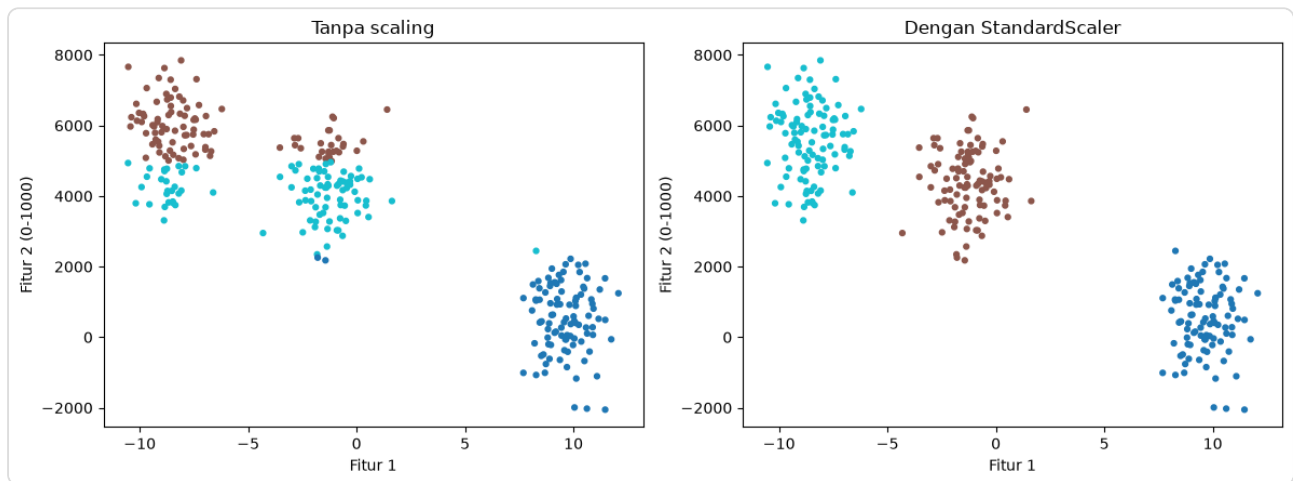
In [5] - Kode Program

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_blobs
4 from sklearn.cluster import KMeans
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import adjusted_rand_score
7
8 # Data 2D dengan skala timpang: fitur1 kecil, fitur2 besar (0-1000)
9 X_pm2, y_asli = make_blobs(n_samples=300, centers=3, random_state=7)
10 X_timpang = X_pm2.copy()
11 X_timpang[:, 1] *= 1000
12
13 label_tanpa = KMeans(n_clusters=3, n_init=10,
random_state=42).fit_predict(X_timpang)
14 label_dengan = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(
15     StandardScaler().fit_transform(X_timpang))
16 print(f"Tanpa scaling : ARI vs label asli = {adjusted_rand_score(y_asli,
label_tanpa):.3f}")
17 print(f"Dengan scaling: ARI vs label asli = {adjusted_rand_score(y_asli,
label_dengan):.3f}")
18
19 # Visualisasi berdampingan
20 fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
21 axes[0].scatter(X_timpang[:, 0], X_timpang[:, 1], c=label_tanpa, cmap="tab10",
s=12)
22 axes[0].set_title("Tanpa scaling")
23 axes[0].set_xlabel("Fitur 1")
24 axes[0].set_ylabel("Fitur 2 (0-1000)")
25 axes[1].scatter(X_timpang[:, 0], X_timpang[:, 1], c=label_dengan, cmap="tab10",
s=12)
26 axes[1].set_title("Dengan StandardScaler")
27 axes[1].set_xlabel("Fitur 1")
28 axes[1].set_ylabel("Fitur 2 (0-1000)")
29 plt.tight_layout()
30 plt.show()

```

Tanpa scaling : ARI vs label asli = 0.570
 Dengan scaling: ARI vs label asli = 1.000



Gambar 3

Penjelasan outputnya:

Tanpa scaling, ARI-nya hanya 0.570: KMeans gagal mengenali struktur asli data. Dari scatter plot kiri terlihat batas cluster-nya nyaris vertikal, artinya pemisahan hanya mengikuti fitur kedua (yang berskala 0-1000) sementara fitur pertama diabaikan sama sekali. Setelah di-scaling, ARI-nya naik menjadi 1.000: cluster yang ditemukan sama persis dengan label asli datanya, dan scatter plot kanannya menunjukkan tiga gugus yang dipisahkan secara wajar di kedua sumbu.

Kesimpulan: tanpa scaling, fitur yang berskala besar mendominasi perhitungan jarak sehingga KMeans seolah-olah hanya melihat satu fitur. StandardScaler menyamakan kontribusi tiap fitur, dan di data ini itu mengembalikan hasil clustering ke struktur aslinya (ARI dari 0.570 menjadi 1.000).

Percobaan 3 - Sensitivitas DBSCAN terhadap eps

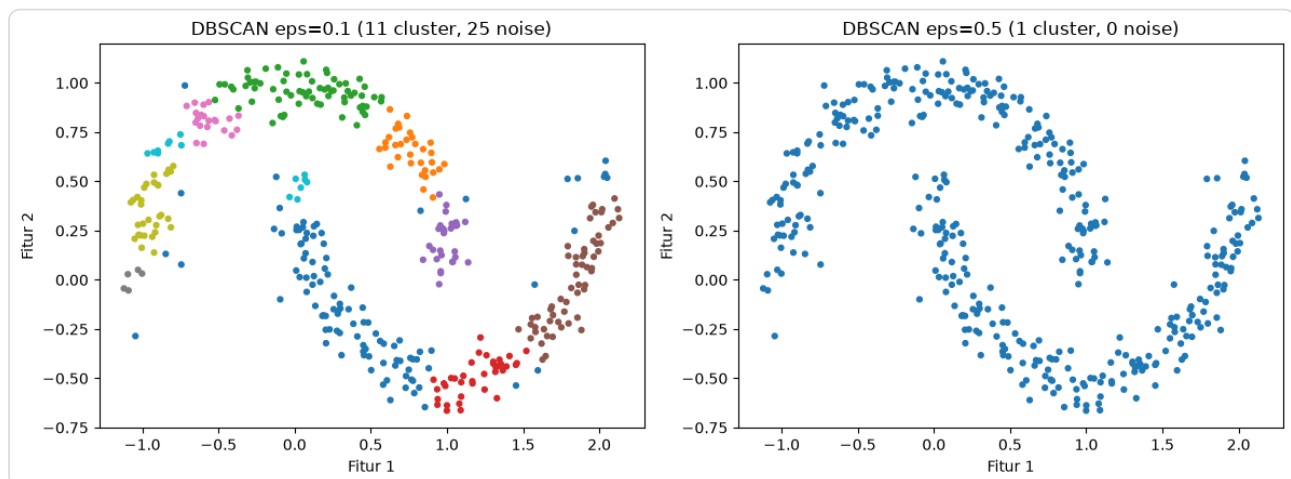
Tujuan: melihat bagaimana parameter eps memengaruhi hasil DBSCAN: terlalu kecil atau terlalu besar, mana yang merusak?

Saya memakai make_moons (400 titik, noise 0.08) karena bentuk bulan sabitnya tidak bisa dipisahkan KMeans secara linear, jadi ini kasus yang cocok untuk DBSCAN. Saya coba dua nilai eps yang ekstrem (0.1 dan 0.5) dengan min_samples=5 yang sama, lalu hitung jumlah cluster dan titik noise-nya.

In [6] - Kode Program

```
1 import matplotlib.pyplot as plt
2 from sklearn.datasets import make_moons
3 from sklearn.cluster import DBSCAN
4
5 # Dua bulan sabit + noise (dibuat mandiri, tidak tergantung sel lain)
6 X_pm3, _ = make_moons(n_samples=400, noise=0.08, random_state=42)
7
8 fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
9 for ax, eps in zip(axes, [0.1, 0.5]):
10     label = DBSCAN(eps=eps, min_samples=5).fit_predict(X_pm3)
11     n_cluster = len(set(label)) - (1 if -1 in label else 0)
12     n_noise = int((label == -1).sum())
13     print(f"eps={eps}: {n_cluster} cluster, {n_noise} titik noise")
14     ax.scatter(X_pm3[:, 0], X_pm3[:, 1], c=label, cmap="tab10", s=12)
15     ax.set_title(f"DBSCAN eps={eps} ({n_cluster} cluster, {n_noise} noise)")
16     ax.set_xlabel("Fitur 1")
17     ax.set_ylabel("Fitur 2")
18 plt.tight_layout()
19 plt.show()
```

eps=0.1: 11 cluster, 25 titik noise
eps=0.5: 1 cluster, 0 titik noise



Gambar 4

Penjelasan outputnya:

Dengan $\text{eps}=0.1$, DBSCAN menemukan 11 cluster dan membuang 25 titik sebagai noise. Dari scatter plot kiri terlihat kedua bulan sabitnya pecah menjadi banyak potongan kecil: karena jangkauan tetangganya terlalu sempit, titik-titik yang sebenarnya satu gugus tidak saling terjangkau. Sebaliknya dengan $\text{eps}=0.5$, DBSCAN hanya menemukan 1 cluster dan 0 noise: jangkauannya terlalu lebar sehingga kedua bulan sabit tersambung menjadi satu gugus raksasa.

Kesimpulan: eps terlalu kecil membuat data terfragmentasi menjadi banyak cluster kecil plus banyak noise, sedangkan eps terlalu besar menyatukan gugus yang berbeda. Nilai eps yang pas untuk data ini ada di antaranya (sekitar 0.2-0.3), jadi pemilihan eps memang perlu di-tuning, bukan asal tebak.

Kesimpulan Bab 10

- Elbow method dan silhouette score bisa dipakai bersama untuk menentukan jumlah cluster: di data ini keduanya mengarah ke $k=5$, dengan siku inertia yang jelas dan silhouette 0.629.
- K-Means dan Agglomerative clustering menghasilkan segmentasi yang setara bagusnya pada data pelanggan ini (silhouette 0.629 vs 0.623, sama-sama 5 cluster tanpa noise).
- DBSCAN menemukan 4 cluster dengan silhouette tertinggi (0.699) tetapi menggabungkan dua segmen dan menandai 31 titik sebagai noise, sehingga kurang pas untuk kebutuhan segmentasi pelanggan.
- Untuk kasus bisnis seperti segmentasi pelanggan mal, algoritma yang menempatkan seluruh data ke segmen yang jelas (K-Means) lebih masuk akal dibanding yang membuang data sebagai noise.